

Foundation and Building Blocks

DENG
Forooz Shahbazi

Information Technology

FH Zentralschweiz

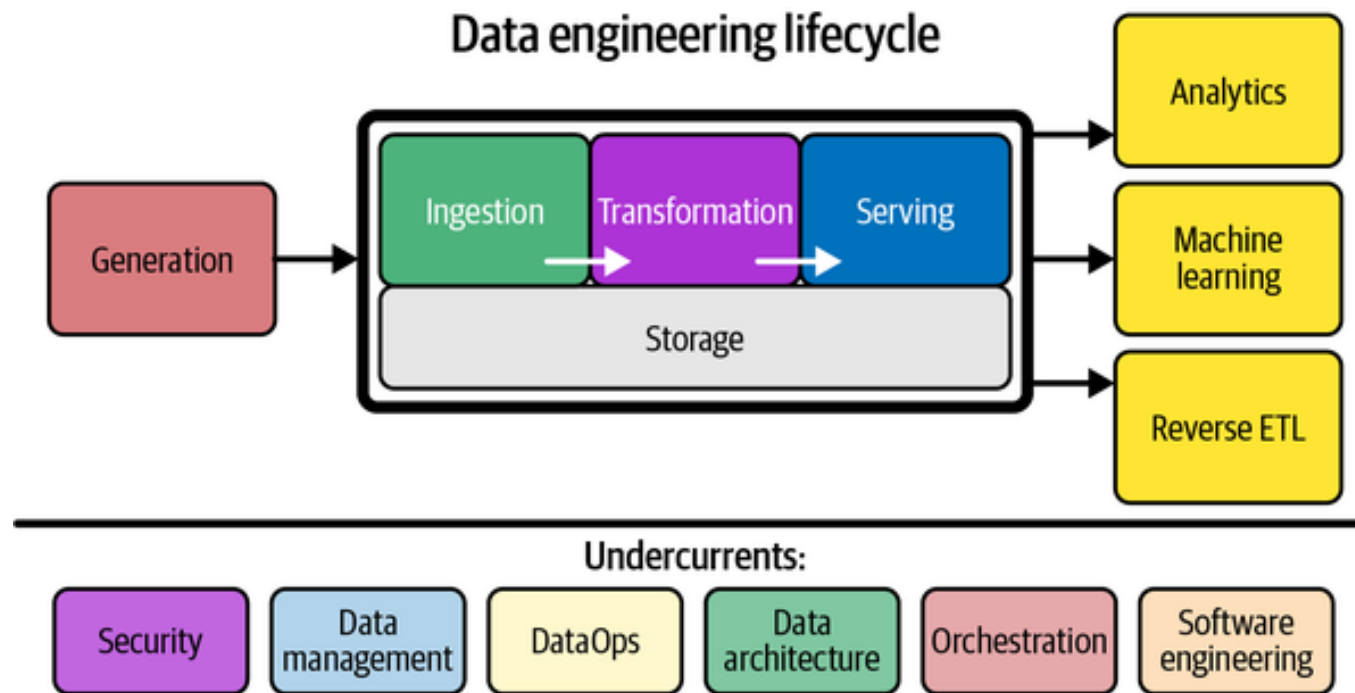
Learning Goals

- Grasp the **data engineering lifecycle** (generation → storage → ingestion → transformation → serving).
- Recognize the **role & skills of data engineers** vs. data scientists.
- Understand **data maturity stages** and how engineering priorities shift.
- Evaluate key lifecycle components: **source systems, storage, ingestion models, metadata & governance**.

Data Engineering Described

Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports downstream use cases, such as analysis and machine learning. Data engineering is the intersection of security, data management, DataOps, data architecture, orchestration, and software engineering. A data engineer manages the data engineering lifecycle, beginning with getting data from source systems and ending with serving data for use cases, such as analysis or machine learning.

The Data Engineering Lifecycle



The Data Engineering Lifecycle

- The data engineering lifecycle shifts the conversation away from technology toward the data itself and the end goals that it must serve. The stages of the data engineering lifecycle are as follows:
 - Generation
 - Storage
 - Ingestion
 - Transformation
 - Serving

Data Engineering and Data Science

- data engineering is *separate* from data science and analytics. They complement each other, but they are distinctly different. Data engineering sits upstream from data science, meaning data engineers provide the inputs used by data scientists (downstream from data engineering), who convert these inputs into something useful.

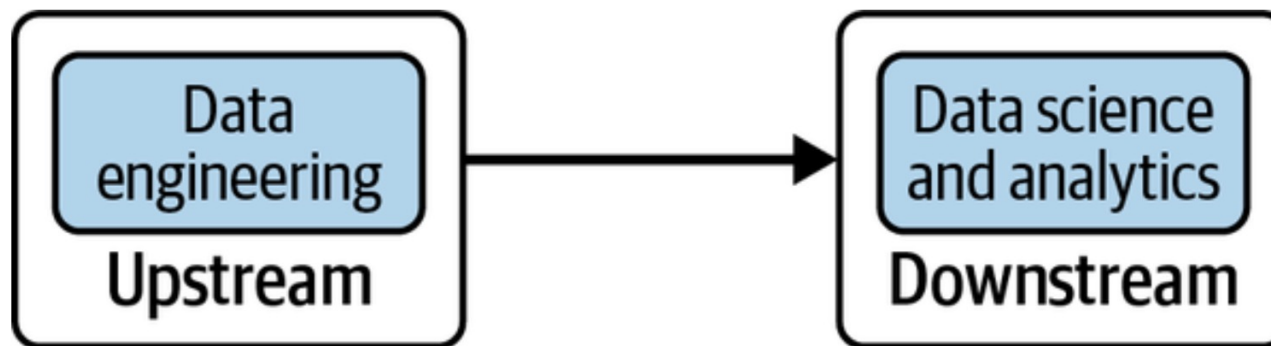
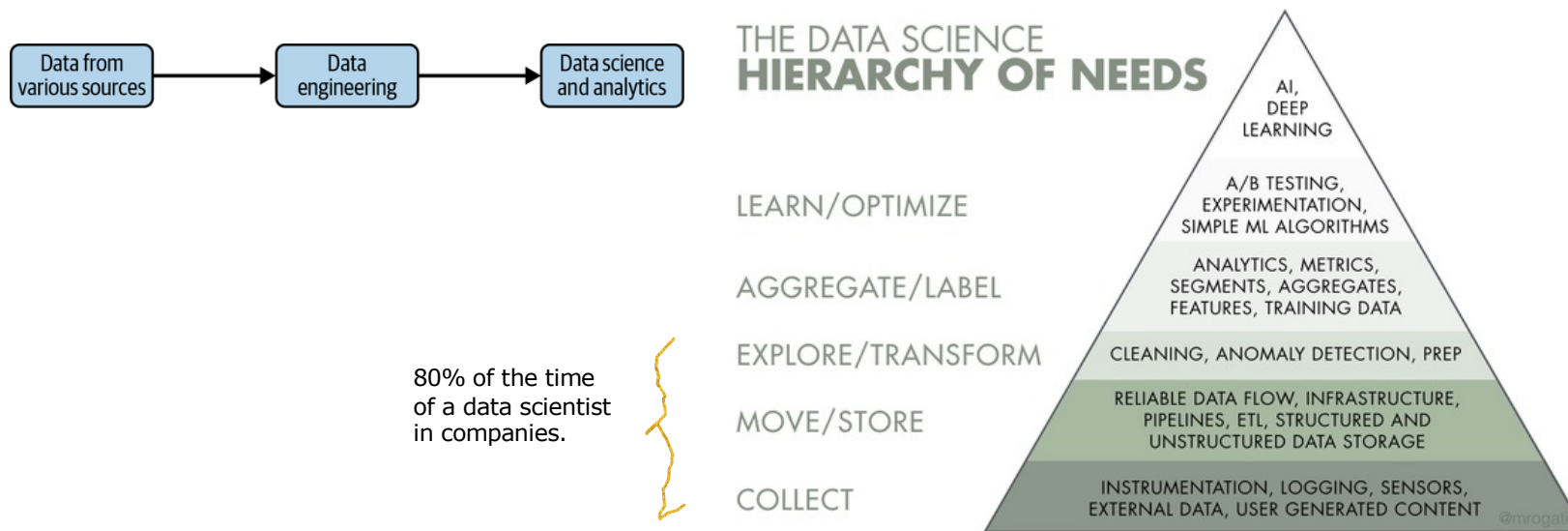


Figure 1-4. Data engineering sits upstream from data science

Data Engineering and Data Science

- In 2017, Monica Rogati published this hierarchy in [an article](#) that showed where AI and machine learning (ML) sat in proximity to more “mundane” areas such as data movement/storage, collection, and infrastructure.



Data Engineering: Skills, Scope & Focus

- **Core Responsibilities:**

- Architect, manage, and optimize the **data lifecycle**: from source to consumption.
- Bridge between data production and analytical value creation.
- Evaluate and integrate tools across security, DataOps, data management, and architecture.

- **Modern Skill Set:**

- **Software engineering foundations** (distributed systems, storage, networking).
- Balance **cost, scalability, simplicity, agility, and interoperability**.
- Use modern, **abstracted tools** (vs. old monolithic systems like Hadoop).

- **What Data Engineers *Don't* Typically Do:**

- Build ML models
- Create dashboards or KPIs
- Conduct in-depth data analysis
- Develop frontend or end-user apps



Figure 1-7. The balancing act of data engineering

Data Engineering Is a Balancing Act

Good architecture balances competing goals — there is rarely one universally “best” solution.

Cost

Infrastructure, licenses, engineering time & maintenance

Example: Streaming may cut latency from 1 h to 5 s — but is the added cost worth it?

Scalability

Ability to handle growth in data, users, queries or events

Example: A script works for a 100 MB CSV. What happens at 1 TB?

Simplicity

Meet requirements without unnecessary technical complexity

Example: If daily batch is enough, Kafka + streaming may be worse engineering.

Agility

How quickly the system can adapt to changing requirements

Example: A new data source is needed: does integration take hours, days or months?

Reuse

Build data/components that can support multiple consumers

Example: Can Finance and ML use the same trusted customer dataset?

Interoperability

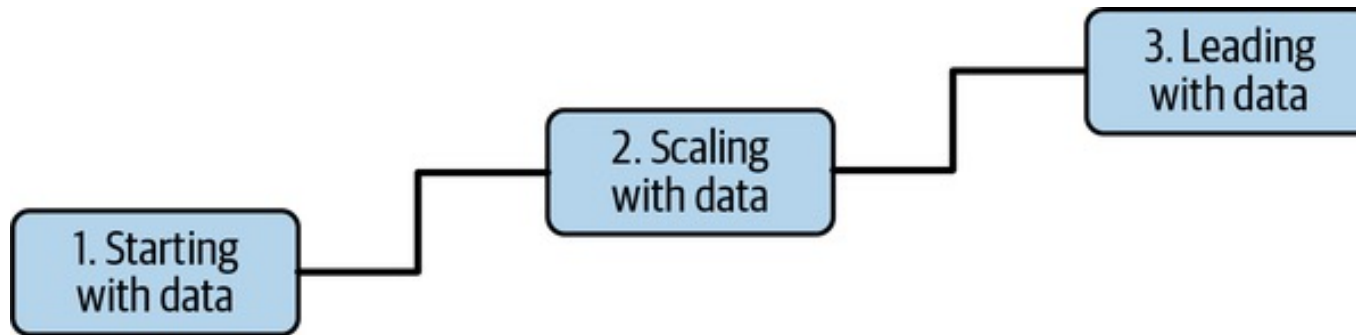
How easily components exchange data and work together

Example: Open formats such as Parquet reduce coupling across tools and platforms.

Key idea: Choose architecture based on requirements and trade-offs — not because a technology is newer or more powerful.

Data Maturity and the Data Engineer

- The level of data engineering complexity within a company depends a great deal on the company's data maturity.
- Data maturity model suggested in the book:



Your Turn!

You're the first data engineer at a 20-person startup.

There are Excel reports, Python scripts and an application database.

The CEO asks you to “build our data platform.”

What are your first three actions?

Stage 1: Starting with Data

Low data maturity

A data engineer should focus on the following in organizations getting started with data.



Goal: Build the right foundation and architecture to turn data into business value.



1. Get buy-in from key stakeholders

Engage key stakeholders, including executive management. Secure a sponsor for critical initiatives to design and build a data architecture that supports the company's goals.



2. Define the right data architecture

Usually done solo, since a data architect likely isn't available. Determine business goals and competitive advantage to achieve with your data initiative. Work toward an architecture that supports these goals.



3. Identify and audit data

Identify and audit data that will support key initiatives and operate within the data architecture you designed.



4. Build a solid data foundation

Build the foundation for future analysts and data scientists to generate reports and models that provide competitive value. In the meantime, you may also have to generate these reports and models until this team is hired.

Stage 2: Scaling with Data



At this point, a company has moved away from ad hoc data requests and has formal data practices in place.

Goals for Data Engineers in Stage 2

- **1 Establish formal data practices**
Define standards, processes, and governance for reliable and consistent data.
- **2 Create scalable and robust architectures**
Design architectures that can grow with the business and handle increasing data volume and complexity.
- **3 Adopt DevOps and DataOps practices**
Automate, monitor, and collaborate to deliver data reliably and iteratively.
- **4 Build systems that support ML**
Provide high-quality, accessible data and features for machine learning at scale.
- **5 Avoid undifferentiated heavy lifting**
Use managed services and customize only when it creates a competitive advantage.



Principle: Leverage managed services and proven tools. Invest your energy where it drives unique business value.



Raw Data



Reliable & Scalable
Data Platform



Business Value
& Insights

Avoid Undifferentiated Heavy Lifting

Focus on building value, not on reinventing infrastructure.



Setting up and managing physical servers

Time-consuming and not a competitive advantage.



Installing and configuring open-source frameworks like Hadoop from scratch.



Manually handling software patches and upgrades for data tools.



Building basic data pipelines that could be easily automated by a pre-built service.

Stage 3: Leading with data



Data-Driven Organization

Automated pipelines and systems enable self-service analytics and ML across the company.



Data is not just available—it drives **decisions** and **innovation**.

In Stage 3, Data Engineers:



1. Create automation for new data

Create automation for the seamless introduction and usage of new data.



2. Build custom tools and systems

Focus on building custom tools and systems that leverage data as a competitive advantage.



3. Focus on “enterprisey” aspects of data

Focus on data management (governance, policies for data use, quality) and DataOps.



4. Expose and disseminate data

Make data easy to find, understand, and access.
Deploy tools like data catalogs, data lineage (origin & transformations), and metadata management systems.



5. Collaborate across disciplines

Collaborate efficiently with software engineers, ML engineers, analysts, and others.



6. Build a data community

Create a community and environment where people can collaborate and speak openly, no matter their role or position.

The Impact



Trusted
Data



Empowered
People



Better Decisions
at Scale



Sustainable
Competitive Advantage

From Zero to Data-Driven: A Data Engineer's Journey

How one data engineer built the data foundation of a startup



Example Startup: **MealBuddy**

A food delivery app connecting local restaurants and customers.



The Data Engineer

Joined as the first (and only) data engineer.



The Mission

Turn raw operational data into insights that drive growth.



The Team (Now)

Data team of 8+ people across data, analytics, and ML.



STAGE 1: STARTING WITH DATA (Low Data Maturity)



The Situation

Data was scattered and ad hoc. Requests came via Slack, reports in Excel, and decisions were based on gut feeling.

What the Data Engineer Did

- Met with key stakeholders and secured executive buy-in.
- Identified key business questions and success metrics.
- Collected data from sources (app DB, payments, logs).
- Built a simple data pipeline (Python scripts + cron).
- Created a central data store (PostgreSQL).
- Built first dashboards to answer top questions.



Key Outcome

Data was centralized and accessible. The team could answer basic questions reliably.



STAGE 2: SCALING WITH DATA (Building Data Practices)



The Situation

The product was growing fast. More data, more users, more questions—manual processes couldn't keep up.

What the Data Engineer Did

- Designed a scalable data architecture on AWS.
- Ingested data using Airflow (scheduled pipelines).
- Moved data to Amazon S3 (data lake).
- Modeled data in dbt and used Redshift.
- Implemented data quality tests and alerts.
- Adopted Git, CI/CD, and monitoring (DataOps).
- Enabled self-service analytics with curated datasets.



Key Outcome

Reliable, scalable, and documented data infrastructure. Teams started doing self-service analytics.

STAGE 3: LEADING WITH DATA (Data-Driven Organization)



The Situation

Data was trusted and widely used. Leadership wanted to leverage data as a strategic advantage.

What the Data Engineer Did

- Automated onboarding of new data sources.
- Built a metrics layer and defined a single source of truth.
- Implemented data governance (policies, access, quality).
- Launched a data catalog (DataHub) and lineage.
- Built reusable data products and feature store for ML.
- Created a data culture: training, docs, office hours.
- Partnered across teams to deliver data-driven features.

What is a Data Catalog?

A data catalog is like a searchable library for all company data. It helps people discover, understand, and trust data assets. Examples: DataHub, Amundsen, Collibra, AWS Glue Data Catalog

What is Lineage?

Lineage shows the origin and journey of data—how it flows through systems and transformations. It answers: Where did this data come from? How was it changed?



Key Outcome

Data became a product and a competitive edge. Decisions are faster, smarter, and measurable.

THE IMPACT OVER TIME



Siloed & inconsistent data



Reliable data & scalable systems



Self-service everywhere



Data drives product & growth



Sustainable competitive advantage



It's not just about tools. It's about solving the right problems, building trust in data, and enabling people to make better decisions.



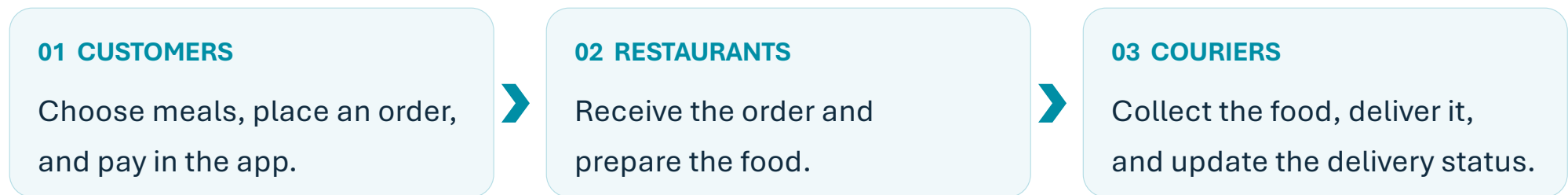
Becoming a Data Engineer: Background & Skills

- **No Standard Path—High Self-Learning Curve**
- Few formal university programs or standard curricula.
- Most data engineers come from adjacent fields:
Software engineering, ETL, DB admin, data science, analytics.
- Success requires **significant self-study** and cross-disciplinary learning.
- **Core Knowledge Areas**
 - **Data fluency:** Data management, governance, quality.
 - **Tech expertise:** Tools evaluation, software engineering, DataOps, architecture.
 - Understand **tool trade-offs** and how systems fit together.
 - **Looking at the big picture, e.g. business goals, user needs, etc.**
 - Must bridge **technical skills** with **business understanding**.
 - Serve as a critical link between **data sources** and **data consumers** (analysts, scientists).

Your Turn!

Meet MealBuddy

A fictional food-delivery platform connecting customers, restaurants, and couriers.



What gets
recorded?

Orders, ordered items, payments, and delivery-status updates.

One order can contain several items and receive several updates.

THE BUSINESS NEED

Every morning, the operations team needs a table of the previous day's orders to understand sales, cancellations, and delivery times.

HSLU Daily output: `orders_per_day` • One row per order

Your Turn!

Make a diagram for the MealBuddy Data Engineering (DE) lifecycle.

Show how data flows from the moment a customer places an order until we have a daily table with **one row per order**.

Requirements:

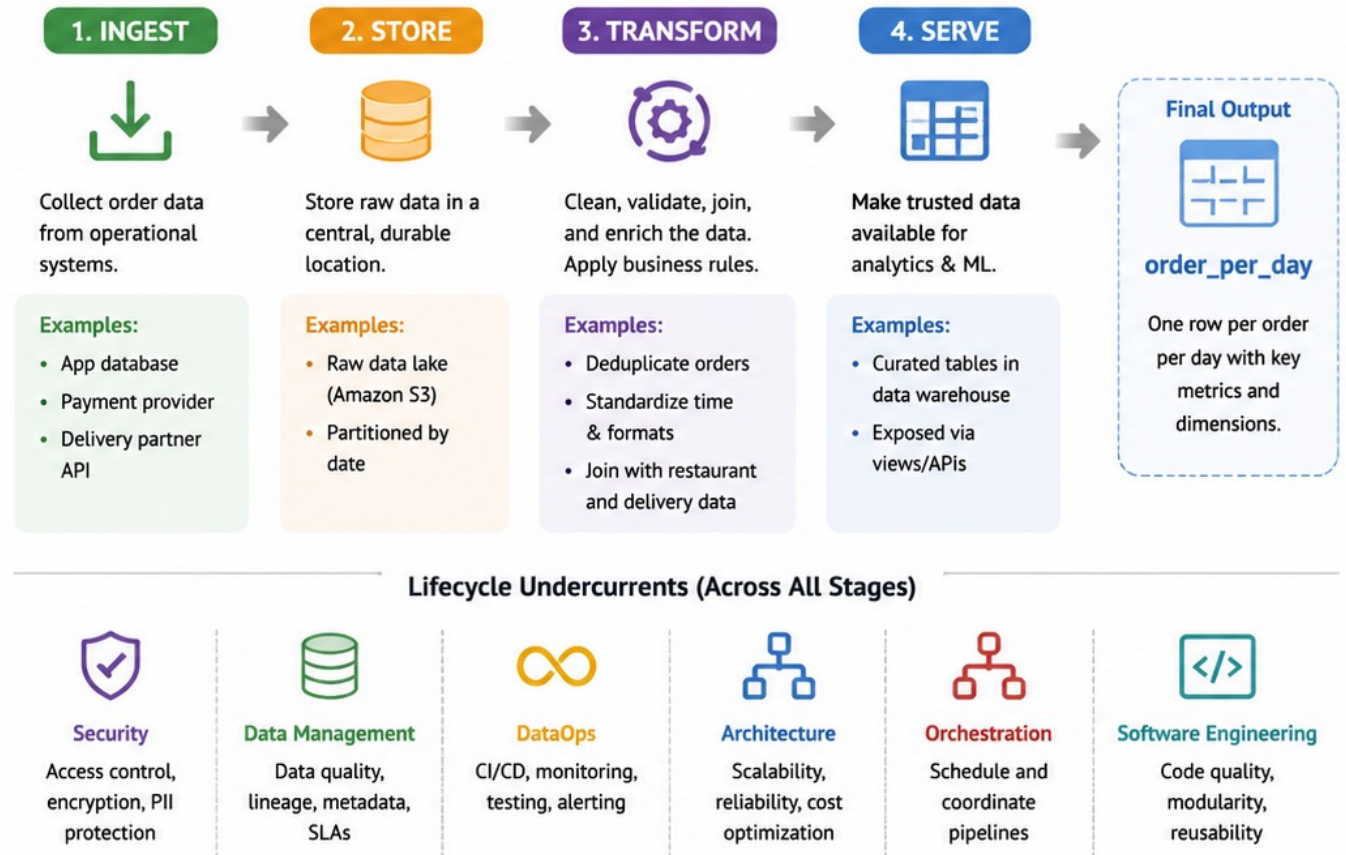
- ✓ Include the main DE lifecycle stages.
- ✓ Show key data stores and data flow.
- ✓ List at least one example of transformation that happens.
- ✓ The final output should be an **order_per_day** table.



Discuss (5 minutes):

Sketch your diagram and be ready to share your design and assumptions.

Example: MealBuddy Data Engineering Lifecycle



Deliverable: A clear diagram of the MealBuddy DE lifecycle from order placement to **order_per_day** table.

We'll discuss a few designs together!

Lifecycle and Lifecycle Undercurrents

- **Core Stages:**
 - **Source → Ingest → Store → Transform → Serve**
 - Data is collected from sources, stored, transformed, and delivered to end users (analysts, scientists, ML engineers).
 -  *These stages may repeat, overlap, or appear out of order—flexibility is expected.*
- **Storage Is Foundational:**

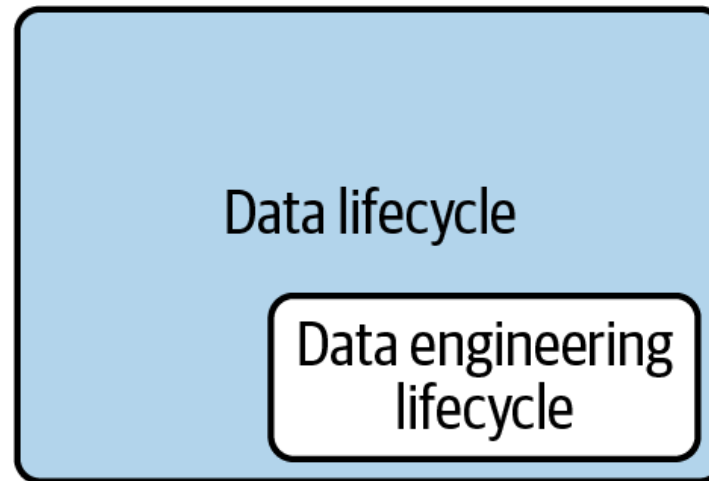
Storage supports the entire lifecycle, not just a single stage — it's the **backbone** of data flow.
- **Lifecycle Undercurrents:**

 Continuous forces that cut across all stages:

 - **Security**
 - **Data Management**
 - **DataOps**
 - **Architecture**
 - **Orchestration**
 - **Software Engineering**

Data Life Cycle Vs. Data Engineering Life Cycle

The data engineering lifecycle is a subset of the whole data lifecycle. Whereas the full data lifecycle encompasses data across its entire lifespan, the data engineering lifecycle focuses on the stages a data engineer controls.



Your Turn: Meet the NYC Taxi Data

- You have joined a team that wants to analyze [taxi trips in New York City](#).
- Before choosing any tools, understand the data:
 - What does **one row** represent?
 - What are the most important fields?
 - Which fields describe the trip, location, time, passenger and payment?
 - Who/what do you think **generates** this data?
 - What potential data-quality problems can you already imagine?
 - What questions could a business or analyst answer with this dataset?

Your Turn: Design the NYC Taxi Data Engineering Lifecycle

- NYC Taxi trip data is generated by the taxi system. Analysts want to explore trips, fares, locations and demand over time.
- Draw a possible data engineering lifecycle:
- **Generation → Ingestion → Storage → Transformation → Serving**
- For each stage, answer:
What happens to the data? Where does it live?
What could go wrong?

NYC Taxi: from a trip to an analytical result

Goal: help analysts explore trip counts, fares, locations, and demand over time.

1 GENERATION	2 INGESTION	3 STORAGE	4 TRANSFORMATION	5 SERVING
WHAT HAPPENS Record pickup, drop-off, locations, and fares. WHERE IT LIVES Taxi / provider systems; then TLC monthly files. WHAT CAN GO WRONG Missing fields or incorrect timestamps.	WHAT HAPPENS Python downloads each monthly Parquet file. WHERE IT LIVES Source website → local landing folder. WHAT CAN GO WRONG Missing file, failed download, schema change.	WHAT HAPPENS Retain source files; load trip records. WHERE IT LIVES Files on disk; PostgreSQL tables on a Docker volume. WHAT CAN GO WRONG Partial loads, duplicate reruns, lost storage.	WHAT HAPPENS Flag issues, join zone names, group by day. WHERE IT LIVES SQL views or derived tables in PostgreSQL. WHAT CAN GO WRONG Wrong join or rule; misleading totals.	WHAT HAPPENS Let analysts query and explore the results. WHERE IT LIVES Reporting views; pgAdmin or a BI tool. WHAT CAN GO WRONG Stale results or incorrect access rights.

Example output: one row per pickup date and zone, with trip count and a defined fare total.

HSLU

Across every stage: data quality, security, monitoring, and clear ownership.

Your Project Starts Here!

Prepare a 5-minute team project pitch for Week 3.
Work with your partner to define your project's foundation.

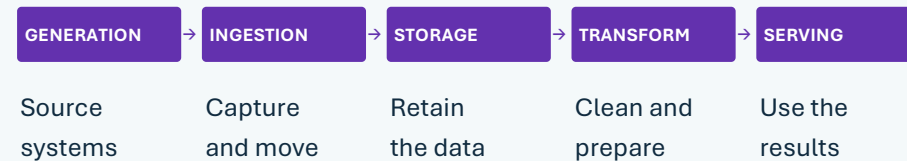
Why this matters

Start with a clear problem, understood data, and a plan for turning data into value.

- 1 Find your partner**
Form your project team.
- 2 Define a meaningful use case**
Start with a problem or question to solve, not just a dataset.
- 3 Find a realistic data source**
Choose an accessible source that preferably changes or receives new data over time.
- 4 Understand your data**
What does one record represent? How often is data generated? How much do you expect?
- 5 Define the consumer and output**
Who will use the data, and what should they be able to do with it?
- 6 Sketch your data engineering lifecycle**
Show the flow from generation to serving.
- 7 Think about what could go wrong**
Identify 2–3 challenges, such as missing data, schema changes, late arrivals, or failures.

Sketch your first data engineering lifecycle

Show how data flows from its source to a useful output.



Examples of challenges

Missing or duplicate records • Schema changes • API limits
Late arrivals • Growing data volume • Pipeline failures

Do not choose technologies yet.

First understand the problem, data, and required flow.

Deliverable for Week 3: a 5-minute project pitch per team

Cover the use case, source, data characteristics, consumer/output, lifecycle sketch, and main challenges.